

ResearchLens - Implementation Plan for the Coding Agent

Based on `researchlens_project.zip` (Streamlit build) and the PDF `research_report_st-57a5744e22.pdf`. Line numbers refer to the files in that zip, before the PDF fix is applied.

0. Read this first

What I verified by running code

- The repeated text in the PDF is a drawing bug, not a report-content bug. I reproduced it by running the real `build_pdf_report` function: one line was drawn 38 times at the same position (in your PDF: 988 text draws over 43 positions, up to 134 on one line).
- A fixed version is done and tested: `researchlens_pdf_fix.zip` (3 files). On the same input it drops to 1 draw per position. 4 new tests pass on the fix, and 2 of them fail on the old code.

What I only read (not run)

- The report-assembly code in `backend/services/local_llm_service.py`, the retrieval code, and the workflow. I did not run the full pipeline or the existing test suite (it needs torch).
- I skimmed the Streamlit worker/lock design but did not audit it line by line.

Important: the PDF was probably made by an older build than this zip

- The PDF says *"No definitive foundational anchor paper was identified in the literature for ..."*. This zip no longer produces that sentence (`_format_first_sentence`, `local_llm_service.py` L260, now returns *"Foundational anchor paper: Not identified for '...'"*).
- This zip also already has a qualification-sentence path in `nli_verifier.py` and an honest-reporting answer (`local_llm_service.py` L296-316).
- **First action:** redeploy this zip (plus the PDF fix), re-run the benchmark, and compare. Some problems seen in the old PDF (anchor contradiction, 33.3% integrity from pruned disclaimer sentences) may already be fixed. Do not re-fix them blindly; confirm each one.

A correction to something I said earlier

- I said the arXiv queries were cut off at a fixed length. That was wrong. The `[ :50]` cut in `paper_retrieval.py` (L424, L428, L489) only shortens the *error message text*. The real queries are not truncated. Query *quality* is still worth improving (section 5).

1. Problems, ranked

ID	Problem	Where	Status
A1	Each word re-draws the growing line at the same spot, so text is overprinted and extracted text is 30x too long	<code>streamlit_app.py</code> <code>render_paragraph</code> (L459-489)	Fixed + tested
A2	Table columns wrong: Dataset shows the year, Metric shows the venue, venue/year cut to 18 chars so the year is lost	<code>streamlit_app.py</code> L497-530 and <code>local_llm_service.py</code> L463-465	PDF side fixed; data side in B1
A3	Bullet and em dash render as <code>?</code> and a middle dot (Helvetica cannot draw them)	<code>streamlit_app.py</code> L479, L401	Fixed (<code>_pdf_safe</code>)
A4	Phantom citation <code>[2]</code> : the PDF appended every citation's badge number even though only 1 reference exists	<code>streamlit_app.py</code> L563-566	PDF side fixed; root cause in B2
B1	Comparison table reuses old fields: <code>dataset=year</code> , <code>f1Score=venue</code> , <code>mapScore=citations</code>	<code>local_llm_service.py</code> L463-465	Open
B2	Citation badge numbers are per citation, but the reference list is per paper, so badges can exceed the reference count	<code>local_llm_service.py</code> L419-433, L636-652	Open

ID	Problem	Where	Status
B3	Raw retrieval errors (HTTP 429, query strings) are printed in Limitations and Conclusion	<code>local_llm_service.py</code> L412, L717; <code>paper_retrieval.py</code> L424, L469, L489, L653	Open
B4	Executive Summary and Findings can contain the same paragraph	<code>local_llm_service.py</code> L656, L727	Open
B5	Hardcoded LLM-inference concept list, section titles and a static technique table (including text like "speedup 2-3x" that does not come from evidence)	<code>local_llm_service.py</code> L584, L666	Open, important
B6	No single source of truth for anchor state	<code>local_llm_service.py</code> L260 and conclusion block (~L690-700)	Open (verify first)
B7	No validation gate before a report is saved or exported	new	Open
C1	Retrieval: oversized Semantic Scholar batch (500 ids), generic queries	<code>paper_retrieval.py</code> L634, L45-92	Open
D1	171 candidates became 1 analyzed paper; cause unknown	<code>research_workflow.py</code> , <code>config.py</code>	Diagnose first
E1	PDF and Markdown are rebuilt on every Streamlit rerun	<code>streamlit_app.py</code> L838	Open (minor)

2. Phase A - apply the verified PDF fix (about 15 min)

1. Copy the 3 files from `researchlens_pdf_fix.zip` into the project:

- `streamlit_app.py` (the PDF code is removed and replaced by `from backend.services.pdf_report import build_pdf_report`)
- `backend/services/pdf_report.py` (new: single-draw wrapping, WinAnsi-safe text, real table fields, citation clean-up)
- `backend/tests/test_pdf_report.py` (new)

2. Run `python -m pytest backend/tests/test_pdf_report.py -q`. Expect 4 passed.
3. Generate a PDF from a real run and run this check. It must report `max == 1`:

```
# scripts/pdf_audit.py
import re, sys, collections
from pypdf import PdfReader
OP = re.compile(r"1 0 0 1 ([\d.\-]+) ([\d.\-]+) Tm\s*/\w+ [\d.]+ Tf[^\[\]]*\[<([0-9a-f]+)>\]TJ")
c = collections.Counter()
for i, pg in enumerate(PdfReader(sys.argv[1]).pages):
    for x, y, _ in OP.findall(pg.get_contents().get_data().decode("latin-1")):
        c[(i, x, y)] += 1
print("draws", sum(c.values()), "positions", len(c), "max",
      max(c.values()))
```

4. Add `pymupdf` and `pypdf` to `requirements-streamlit.txt` / test requirements if missing.
5. Do **not** change anything in report generation for the repetition problem. The Markdown was never the cause.

3. Phase B - report content correctness

Do these one at a time. After each, run the new test for it and the PDF audit.

B1. Stop reusing legacy table fields (`local_llm_service.py` L438-466)

- In `build_comparison_table`, `dataset=year_str`, `f1Score=venue_str`, `mapScore=cites_str` are the cause of the wrong columns.
- Before changing, search for every reader of `row.dataset`, `row.f1Score`, `row.mapScore` (`grep -rn "f1Score\|mapScore\|\.dataset" streamlit_app.py backend src`). Check `build_markdown_report` (L185) and `build_html_report` (L258) too.
- Change the writer to `dataset="Not extracted"`, `f1Score="Not extracted"`, `mapScore="Not extracted"`, and make every reader use `title`, `authors`, `year`, `venue`, `citationCount`.
- Test: build a table row for a paper with year 2023 and venue "X" and assert the Markdown, HTML and PDF show year under Year and venue under Venue.

B2. One numbering system for citations (L419-433 , L636-652)

- Today `[n]` is `Citation.badgeNumber` (one per evidence claim), while the reference list has one entry per paper. Two claims from one paper give `[1]` and `[2]` but only reference `[1]`.
- Change: after `references = build_references_from_citations(...)`, build `ref_index = {paper_id: position + 1}`. When rendering any `[n]` (summary, findings, honest answer at L307, L313, L322), use `ref_index[citation.paperId]`. Collapse repeats such as `[1][1]`.
- Keep `badgeNumber` only for the evidence panel in the UI.
- Test: build a report with 2 claims from 1 paper. Assert every `[n]` in all text satisfies `1 <= n <= len(references)`.

B3. Keep raw errors out of prose (L412 , L717 ; workflow L348, L580, L711-728)

- Retrieval errors are strings like `Semantic Scholar HTTP 429 rate limit for query '...'`. They flow into `retrieval_warnings`, then into Limitations (`retrieval_warnings[:3]`, L412) and the Conclusion (`warning_notes.extend(retrieval_warnings)`, L717).
- Add `summarize_source_status()` that uses the existing `get_source_status()` (`paper_retrieval.py` L235) and returns plain lines such as `Semantic Scholar: rate limited; arXiv: partial results; OpenAlex: ok`.
- In `build_limitations_from_stats` replace the `retrieval_warnings` loop with one Limitations line from that function. **Remove** the `warning_notes.extend(retrieval_warnings)` line from the Conclusion entirely.
- Keep the raw strings in `debug_info["retrieval_warnings"]` only (logs/debug panel).
- Test: simulate a 429 and assert no report text contains `HTTP`, `429`, or `all:"`.

B4. Executive Summary must not copy Findings (L656 , L727)

- `executiveSummary=verified_answer` (L727), and the "Verified Findings & Attribution" section is also `verified_answer` (L656). That is why the PDF shows the same paragraph twice.
- Change the summary to at most 3 sentences: the coverage statement, then one short sentence per concept with verified evidence. Findings contain only per-concept claim text.
- Skip the generic "Verified Findings" section when its text would repeat the summary.
- Test: no sentence longer than 40 characters appears in both summary and findings.

B5. Remove hardcoded LLM-inference content (L584 , L666) - important

- `target_concept_defs` (L584) lists 10 LLM-inference concepts with keywords. The finding section titles ("Memory Management & KV-Cache Optimization", etc.) are also hardcoded. For any other research question this produces wrong coverage lines and wrong headings.
- `tech_comparison` (L666) is a static table with fixed claims such as "Draft model accelerates autoregressive decoding speedup 2-3x". These are not from retrieved evidence, which breaks the "claims only from evidence" rule.
- Change:
 1. Read `backend/services/query_planner.py` to see what the planner outputs. If it does not output facets (the aspects the question names), add a facet-extraction step (LLM with a keyword fallback).
 2. Build the concept list and section titles from those facets. Mark a concept FOUND only if a VERIFIED citation's claim or passage matches it.
 3. Build technique-table rows only for concepts with evidence, with the impact text taken from the verified claim. Omit rows with no evidence and list them under "Insufficient evidence in retrieved corpus".
- Check where `technique_comparison` and `concept_coverage` are displayed (`grep -rn technique_comparison streamlit_app.py src`) and make sure the UI handles an empty table.
- Test: run two unrelated questions (for example one on a different ML topic and one non-ML). Assert no LLM-inference words appear in their output.

B6. Anchor state in one place (L260 and the conclusion block near L690-700)

- The old PDF said "not identified" in one section and "confidence is high" in another. This zip uses the same condition in both places, so it may already be consistent. Do not change anything until the new benchmark run shows a contradiction.
- Either way, add `anchor_state(anchor_paper, anchor_confidence) -> (label, sentence)` and have `_format_first_sentence`, `build_methodology_from_stats`, `build_limitations_from_stats`, and the conclusion all call it. If `anchor_paper` is `None`, confidence is `NONE` everywhere.
- Test: with `anchor_paper=None` and `anchor_confidence="high"`, assert no text says "confidence is high".

B7. Validation gate (new `backend/services/report_validator.py`)

- `validate_report(report) -> list[str]` returns problems. Checks:
 - the same sentence longer than 40 characters in more than one section

- repeated paragraphs
- any `[n]` outside `1..len(references)`
- empty sections and headings with no content
- raw error strings (`HTTP` , `429` , `all:"` , `Traceback`)
- anchor contradiction
- claims without a citation
- Call it in `research_workflow.py` just before the report is saved, and again in `build_pdf_report` . On failure, set the job to a clear `generation_error` status with the problem list. Do not produce a PDF.
- One unit test per check, using sentences from the bad report as fixtures.

B8. Coverage warning for any question

- The honest-reporting answer (`L296-316`) only runs for LLM-inference questions (`is_llm_inference_optimization_query`) or questions containing "technique". Change it to apply to any literature-review question with at most 1 distinct evidence-bearing paper.
- Required wording when evidence-bearing papers are 1 or fewer: *"Research coverage is insufficient to answer this question in general. Only N relevant paper(s) with usable evidence were retrieved."*

4. Phase C - metrics

Keep `citation_integrity` and `evidence_coverage` . Add and show: `research_depth` , `concept_coverage` (found / total from B5), `relevant_papers` , `evidence_bearing_papers` . A high evidence coverage must never be shown as research completeness. Compute `research_confidence` from these numbers.

5. Phase D - retrieval

Already present (do not rebuild): per-source throttling and Retry-After handling, tenacity retries for Semantic Scholar, per-source status (`set_source_status`), a disk response cache, and OpenAlex fallback.

Gaps to fix

1. `enrich_papers_with_s2` sends batches of 500 ids (`paper_retrieval.py` L634). Use 100 and wait about 1 second between batches. A single 500-id call is what produced "429 for 171 ids".

2. Query generation (L45-92): Gemini returns 3 generic queries, or the fallback returns one long stop-word-stripped string. Use planner facets instead: 3-4 topical queries per facet, deduplicated, with source-specific syntax built in one place. Reject queries made only of generic words.
3. Error text includes the query (L424 , L489 , L950). Keep that in logs only.
4. On Streamlit Community Cloud the IP is shared, so unauthenticated Semantic Scholar and arXiv calls get limited quickly. Set `SEMANTIC_SCHOLAR_API_KEY` and an OpenAlex contact email in Streamlit secrets.
5. The disk cache is lost when the app restarts. Accept that, but mention it in `DEPLOY_STREAMLIT.md`.

Tests: mock a 429 and assert the retry count, that the job still finishes with data from another source, and that source status is recorded.

6. Phase E - find where 171 candidates became 1 paper

1. Stage stats already exist (`record_stage` , `StageStat`). Show them as a funnel in the UI and in Limitations: discovered, unique, relevant (after reranking), full text obtained, passages extracted, evidence-bearing, claims verified.
2. Run the benchmark and read which stage lost most papers. Likely suspects: relevance cutoffs (`config.py` L57 `MIN_PAPER_RELEVANCE = 0.40` , L76 `RELEVANCE_THRESHOLD = 0.30`), PDF download failures, the evidence gate.
3. Change only the stage the funnel blames. **Do not lower** `NLI_ENTAIL_THRESHOLD` or any verification threshold.
4. Confirm `routers/research.py` and `config.DEPTH_COUNTS` (L93) agree on paper counts per depth (an earlier version did not).

7. Phase F - Streamlit (minor)

- `streamlit_app.py` L838-839 rebuilds the PDF and Markdown on every rerun. Cache them per job in `st.session_state` , keyed by job id and `updatedAt` .
- The background worker and `_PIPELINE_LOCK` (L119, L601, L636) look reasonable by design. Make sure a second click while a run is active is ignored and each run writes only to its own job record.

8. Order of work and acceptance

1. A (PDF fix), then redeploy and re-run the benchmark. Save the new PDF and Markdown.

2. B1, B2, B3, B4 (visible content bugs), then B7 (gate) so regressions are caught.
3. B5 and B8 (remove hardcoding, general coverage warning), then B6 if still needed.
4. E (funnel) before touching any threshold, then D (retrieval).
5. C (metrics) and F (Streamlit).

Acceptance for the benchmark question and two unrelated questions

- `pdf_audit.py` shows max 1 draw per position, and extracted PDF text is about the size of the Markdown.
- No `HTTP`, `429`, or query strings in any report text.
- Every `[n]` resolves to a reference.
- Anchor statements agree in every section.
- Summary and findings do not repeat each other.
- Output for the unrelated questions has no LLM-inference wording.
- When evidence-bearing papers are 1 or fewer, the coverage warning is present.
- `pytest backend/tests` and `python -m pyflakes backend streamlit_app.py` pass.

Report for each run: papers discovered, unique, relevant, full-text, evidence-bearing, claims verified/contradicted/insufficient, concept coverage, citation integrity, evidence coverage, research depth, confidence, and per-source status.

9. Rules for the agent

- Do not add sources, lower verification thresholds, or invent claims.
- Do not hardcode any question's topics, queries, or concepts.
- Make the smallest change that fixes each item, add the test for it first, and list every file you change.
- If something cannot be fixed within Streamlit Community Cloud limits (memory or persistent 429s), say so plainly and propose a small hosted backend instead.